🎬 MovieCity — Landing Page

Landing page responsiva de um serviço de streaming fictício, construída com **React + Vite**. O objetivo do projeto é ser um exercício prático de componentização, estilização com CSS puro e adaptação de layout para diferentes tamanhos de tela.



📌 Sumário

1. O que o projeto faz
2. Tecnologias utilizadas
3. Pré-requisitos
4. Passo a passo para desenvolver
5. Estrutura de pastas
6. Como as partes se conectam
7. Responsividade
8. Scripts disponíveis
9. Próximos passos / ideias

🎯 O que o projeto faz

A página é dividida em duas áreas principais:

- **Header (cabeçalho):** logo "MovieCity", menu de navegação (Filmes, Categoria, Sobre, Contato) e um botão de **Login**.
- **Main (conteúdo principal):** uma coluna com texto de chamada ("Seus Filmes") + botão **Mais**, e outra coluna com um **grid de 6 cards** de filmes (Afterlife, Transformers, Mowgli, Shift, Round 6 e Harry Potter).

Em telas pequenas, as duas colunas se empilham, o grid de cards passa a ter 2 colunas e o menu de navegação vira um **menu hambúrguer** que abre e fecha.

🧰 Tecnologias utilizadas

Tecnologia	Para que serve
React 19	Biblioteca de UI baseada em componentes
Vite 7	Servidor de desenvolvimento com HMR e bundler de produção
JavaScript (ESM)	Linguagem e sistema de módulos
CSS puro	Estilização global e responsividade (media queries)
ESLint	Padronização e análise estática do código

> Nenhuma biblioteca de UI (Bootstrap, Tailwind, styled-components) é usada de propósito: o objetivo é praticar CSS de verdade.

✅ Pré-requisitos

Antes de começar você precisa ter instalado:

- **Node.js 20.19+ ou 22.12+** (o Vite 7 exige essas versões) — baixar aqui
- **npm** (já vem junto com o Node.js) — ou `yarn`/`pnpm`, se preferir
- Um editor de código, como o VS Code

Confira as versões instaladas:

```
node -v
npm -v
```

🚀 Passo a passo para desenvolver

Passo 1 — Criar o projeto React com Vite

Na pasta onde você guarda seus projetos, rode:

```
npm create vite@latest 01_landinpage -- --template react
```

Durante a criação escolha **React** e depois **JavaScript** (não use TypeScript, este projeto é em JS).

> Se o terminal perguntar se deseja instalar o pacote `create-vite`, responda **yes**. O `--` extra é necessário para que as flags sejam repassadas ao gerador, e não ao npm.

Passo 2 — Entrar na pasta e instalar as dependências

```
cd 01_landinpage
npm install
```

Isso baixa o React, o React DOM e as ferramentas de desenvolvimento listadas no `package.json`.

Passo 3 — Subir o servidor de desenvolvimento

```
npm run dev
```

O Vite abre em `http://localhost:5173`. Deixe esse terminal rodando: cada vez que você salvar um arquivo, a página atualiza sozinha (HMR — *Hot Module Replacement*).

Passo 4 — Limpar o template

O template padrão do Vite traz um contador de exemplo. Para deixar o projeto limpo:

1. Abra `src/App.jsx` e remova o conteúdo de exemplo (contador, logos, links).
2. Apague as imagens `src/assets/react.svg` e `public/vite.svg` apenas se não for usá-las (a última é o favicon definido no `index.html`).
3. Mantenha o `src/main.jsx` como está — é ele que monta o React na `<div id="root">` do `index.html`.

Passo 5 — Criar a pasta de componentes

Dentro de `src`, crie a pasta `components`:

```
mkdir src/components
```

Passo 6 — Criar o componente `Home.jsx`

O `Home` é o container da página: ele junta o cabeçalho e o conteúdo principal.

```
import React from "react";
import Header from "./Header";
import Main from "./Main";

const Home = () => {
    return (
        <div className="home">
            <Header />
            <Main />
        </div>
    );
}

export default Home;
```

Passo 7 — Criar o componente `Header.jsx`

O cabeçalho tem três partes: logo, navegação e ações (Login + botão do menu). O estado `menuOpen` controla a abertura do menu no mobile.

```
import React, { useState } from "react";

const links = [
    { label: "Filmes", href: "/" },
    { label: "Categoria", href: "/" },
    { label: "Sobre", href: "/" },
    { label: "Contato", href: "/" },
```

```
];

const Header = () => {
    const [menuOpen, setMenuOpen] = useState(false);
    const closeMenu = () => setMenuOpen(false);

    return (
        <header className="header">
            <div className="logo">
                <h1>MovieCity</h1>
            </div>

            <nav className={`navigation ${menuOpen ? "open" : ""}`}>
                <ul>
                    {links.map((link) => (
                        <li key={link.label}>
                            <a href={link.href} onClick={closeMenu}>{link.label}
</a>
                        </li>
                    ))}
                </ul>
            </nav>

            <div className="header-actions">
                <div className="btn">
                    <button type="button">Login</button>
                </div>
                <button
                    type="button"
                    className={`menu-toggle ${menuOpen ? "active" : ""}`}
                    aria-label={menuOpen ? "Fechar menu" : "Abrir menu"}
                    aria-expanded={menuOpen}
                    onClick={() => setMenuOpen((prev) => !prev)}
                >
                    <span></span>
                    <span></span>
                    <span></span>
                </button>
            </div>
        </header>
    );
}

export default Header;
```

Pontos importantes:

- A lista de links fica em um array (`links`) e é renderizada com `.map()`, evitando repetição.
- A classe `open` é adicionada/removida conforme o estado — o **CSS** é quem decide o que "abrir" significa.
- Os três `<span>` são as barrinhas do ícone hambúrguer (o ícone é feito só com CSS).

Passo 8 — Criar o componente `Main.jsx`

```
import React from "react";

const Main = () => {
    return (
        <div className="main">
            <div className="col col1">
                <h2>Seus Filmes</h2>
                <p>Aqui, você pode escolher o filme que deseja assistir.<br />
                    Caso queira escolher mais, clique no botão.
                </p>
                <button type="button">Mais</button>
            </div>
            <div className="col cards">
                <div className="card card1"></div>
                <div className="card card2"></div>
                <div className="card card3"></div>
                <div className="card card4"></div>
                <div className="card card5"></div>
                <div className="card card6"></div>
            </div>
        </div>
    );
}

export default Main;
```

Passo 9 — Adicionar as imagens dos cards

Coloque as imagens dos pôsteres em `src/assets/`. Neste projeto são:

```
src/assets/
├── afterlife.jpg
├── transformers.jpg
├── mowgli.jpg
├── shift.png
├── squidgame.webp
└── harrypotter.jpg
```

Como o CSS está em `src/index.css`, o caminho relativo é `url(assets/nome-da-imagem.ext)`.

Passo 10 — Ligar o `Home` no `App.jsx`

```
import './index.css';
import Home from './components/Home';

export default function App() {
```

```
    return (
        <div>
            <Home />
        </div>
    );
}
```

Passo 11 — Escrever o CSS global (`src/index.css`)

Aqui está o coração do projeto. A ordem recomendada é:

1. **Reset** — remove margens/paddings padrão e aplica `box-sizing: border-box`. O `box-sizing` é o que impede que padding e borda "estourem" a largura dos elementos.
2. **Layout base** — `.home` como coluna flex de altura mínima `100vh`.
3. **Blocos** — `.header`, `.main`, `.cards`, `.card`.
4. **Media queries** — no final, ajustando o que já existe.

Trechos-chave do layout:

```
/* A página ocupa a tela toda e empilha header + main */
.home {
    min-height: 100vh;
    background-color: #000;
    display: flex;
    flex-direction: column;
}

/* O main preenche o espaço que sobra abaixo do header */
.main {
    display: flex;
    flex: 1;
    align-items: center;
    gap: 48px;
}

/* Grid que se adapta sozinho à largura disponível */
.cards {
    display: grid;
    grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 20px;
}
```

Passo 12 — Conferir no navegador

1. Abra `http://localhost:5173`.
2. Abra o **DevTools** (`F12`) e ative o **Device Toolbar** (`Ctrl + Shift + M`).
3. Teste nas larguras **1440px**, **1024px**, **768px**, **600px** e **375px**.
4. Confirme que não aparece barra de rolagem horizontal em nenhuma delas.

Passo 13 — Lint e build

Antes de considerar pronto, garanta que o código está limpo e que o build funciona:

```
npm run lint
npm run build
```

O build gera a pasta `dist/`, pronta para publicar em qualquer host estático (Netlify, Vercel, GitHub Pages, etc.). Para pré-visualizar o resultado:

```
npm run preview
```

📁 Estrutura de pastas

```
01_landinpage/
├── public/
│   └── vite.svg              # favicon
├── src/
│   ├── assets/               # imagens dos pôsteres
│   ├── components/
│   │   ├── Header.jsx        # cabeçalho + menu hambúrguer
│   │   ├── Main.jsx          # texto de chamada + grid de filmes
│   │   └── Home.jsx          # container da página
│   ├── App.jsx               # renderiza <Home />
│   ├── App.css               # sobras do template (não usado)
│   ├── index.css             # estilos globais + responsividade
│   └── main.jsx              # ponto de entrada do React
├── index.html                # HTML raiz (lang, título, metadados)
├── eslint.config.js
├── package.json
└── vite.config.js
```

🔗 Como as partes se conectam

```
index.html  →  main.jsx  →  App.jsx  →  Home.jsx  ─┬→  Header.jsx
                                                   └→  Main.jsx
```

1. `index.html` contém apenas `<div id="root">` e o script de entrada.
2. `main.jsx` usa `createRoot()` para renderizar o `<App />` dentro dessa div.
3. `App.jsx` importa o CSS global e renderiza `<Home />`.
4. `Home.jsx` monta `<Header />` e `<Main />`.

5. Os componentes usam `className`, e todo o visual vem de `src/index.css`.

📱 Responsividade

O layout foi construído **mobile-friendly desde a base** e usa quatro breakpoints:

Breakpoint	Alvo	O que muda
`max-width: 1100px`	Tablets em paisagem	Espaçamentos, gaps e tipografia menores
`max-width: 900px`	Tablets retrato / celulares grandes	Colunas empilham, texto centralizado, **menu hambúrguer** aparece
`max-width: 600px`	Celulares	Grid de cards passa para **2 colunas**, fontes e paddings reduzidos
`max-width: 380px`	Celulares pequenos	Ajustes finos de fonte e altura dos cards

Técnicas usadas:

- **Unidades relativas** (`%`, `minmax(0, 1fr)`, `max-height`) em vez de larguras fixas, exceto alturas de card, que são controladas por breakpoint.
- **Flexbox** para as colunas do `main` e para o alinhamento do header.
- **CSS Grid** com `minmax(0, 1fr)` para os cards — o `minmax(0, ...)` evita que o conteúdo force a coluna a crescer além do container.
- **Menu hambúrguer só com CSS**: no desktop a `nav` é normal; abaixo de 900px ela vira um painel posicionado com `position: absolute` e animado via `max-height`.
- **`min-height: 100vh`** no `.home` em vez de `height: 100vh`, para o conteúdo poder crescer sem ser cortado em telas baixas.
- **Acessibilidade**: o botão do menu tem `aria-label` e `aria-expanded`, e o menu fecha ao clicar em qualquer link.

Para testar rapidamente as larguras no navegador:

```
F12 → Ctrl + Shift + M → escolha "iPhone 12 Pro" / "iPad Mini" / "Responsive"
```

📜 Scripts disponíveis

Comando	Descrição
`npm run dev`	Inicia o servidor de desenvolvimento com HMR em `http://localhost:5173`
`npm run build`	Gera a versão otimizada de produção em `dist/`
`npm run preview`	Serve localmente o build de `dist/` para conferência

Comando	Descrição
`npm run lint`	Roda o ESLint em todo o projeto

💡 Próximos passos / ideias

- Substituir os `div`s dos cards por botões `<button>` para melhorar a acessibilidade e o teclado.
- Dar título e nota a cada filme, deixando os cards mais informativos.
- Buscar a lista de filmes de uma API (ex.: TMDB) e renderizar os cards dinamicamente com `.map()`.
- Transformar os cards em um carrossel horizontal no mobile.
- Adicionar `prefers-reduced-motion` para desativar as animações de hover em quem preferir.
- Migrar o CSS para CSS Modules ou para uma solução por componente conforme o projeto crescer.